ReRoute.

Backend Walkthrough

Five short code pages for the presentation. Illustrative snippets and sample outputs. This viewer does not change or operate the travel website.

01 - Request flow
02 - Group access
03 - Demand availability
04 - Toll-based ETA
05 - Data model

From a request to a response.

The frontend sends a request. The server checks identity, reads the database and returns the data the user is allowed to see.

01 - Request flow | Presentation examples | Request handling

File: api/groups.js

```
// Illustrative server handler. Helpers are abbreviated.
async function getGroups(request, database) {
  const userId = trustedIdentity(request);
  const inviteCode = new URL(request.url)
    .searchParams.get("code");

  const groups = await listVisibleGroups(database, {
    userId,
    inviteCode
  });

  return Response.json({ groups, userId }, {
    headers: { "Cache-Control": "no-store" }
  });
}
```

> Explain: the UI does not read the database directly. A server controls which groups and member details are returned. Identity must come from the trusted hosting layer.

SAMPLE OUTPUT - NOT A LIVE SERVER RESPONSE

```
{
  "groups": [
    {
      "name": "Demo Chennai crew",
      "count": 4,
      "minimum": 4
    }
  ],
  "userId": "demo-user"
}
```

The right people, in the right group.

A private invite lets someone request access. Only an accepted member can view the passenger list for that group.

02 - Group access | Presentation examples | Membership checks

File: services/group-access.js

```
// Proposed tracking access guard, using server identity.
async function getCrew(db, groupId, userId) {
  if (!userId) throw new Error("Sign in first");

  const member = await db.prepare(
    "SELECT status FROM members " +
    "WHERE group_id = ? AND user_id = ?"
  ).bind(groupId, userId).first();

  if (member?.status !== "accepted") {
    throw new Error("Group membership required");
  }

  return db.prepare(
    "SELECT name FROM members " +
    "WHERE group_id = ? AND status = 'accepted'"
  ).bind(groupId).all();
}
```

Explain: an invite code and authenticated membership are different. This is a production access example. The website's REROUTE26 tracking code uses fictional presentation data.

SAMPLE OUTPUT - NOT A LIVE SERVER RESPONSE

```
{
  "group": "Demo Chennai crew",
  "acceptedMembers": [
    "Arjun",
    "Meera",
    "Kavin",
    "Priya"
  ],
  "access": "accepted member only"
}
```

Make the capacity gap visible.

Compare three times the normal demand with half the normal supply, then account for extra trips, shifted travel and reserved seats.

03 - Demand availability | Presentation examples | Capacity calculation

File: services/demand.js

```
function planCapacity(normalSeats, extraTrips = 0,
                      shiftPercent = 0, release = false) {
  const demand = normalSeats * 3;
  const supply = Math.floor(normalSeats / 2)
               + extraTrips * 40;
  const shifted = Math.round(demand * shiftPercent / 100);
  const reserve = Math.floor(supply * 0.10);
  const available = supply - (release ? 0 : reserve);
  const remaining = demand - shifted;

  return {
    demand, available, shifted,
    accommodated: Math.min(available, remaining),
    waitlisted: Math.max(0, remaining - available)
  };
}

planCapacity(1000, 5, 20, true);
```

> Explain: 5 extra trips add 200 seats. Moving 20% of demand shifts 600 passengers to other dates. The remaining 1,700 still need alternatives. This does not book seats.

SAMPLE OUTPUT - NOT A LIVE SERVER RESPONSE

```
{
  "demand": 3000,
  "available": 700,
  "shifted": 600,
  "accommodated": 700,
  "waitlisted": 1700
}
```

A checkpoint gives us a new estimate.

Calculate an average speed from two confirmed toll timestamps. Use the next segment's road distance to estimate travel time.

04 - Toll-based ETA | Presentation examples | Milestone forecast

File: services/eta.js

```
function forecast(distanceAB, elapsedMinutes, distanceBC,
                  confirmedStopMinutes = 0) {
  if (![distanceAB, elapsedMinutes, distanceBC,
        confirmedStopMinutes].every(Number.isFinite) ||
      distanceAB <= 0 || elapsedMinutes <= 0 ||
      distanceBC < 0 || confirmedStopMinutes < 0) {
    throw new Error("Invalid segment input");
  }

  const speedKmh = distanceAB / (elapsedMinutes / 60);
  if (speedKmh > 130) throw new Error("Review check-in");

  const etaMinutes = distanceBC / speedKmh * 60
                   + confirmedStopMinutes;
  return { speedKmh, etaMinutes, position: "estimated" };
}

forecast(60, 60, 40, 45); // Reported 45-minute stop
```

Explain: 60 km in 60 minutes gives 60 km/h. The next 40 km takes 40 minutes, plus the reported 45-minute stop. FASTag alone cannot identify stops or highway exits. The 130 km/h threshold is illustrative.

SAMPLE OUTPUT - NOT A LIVE SERVER RESPONSE

```
{
  "speedKmh": 60,
  "etaMinutes": 85,
  "position": "estimated"
}
```

Keep observations separate from forecasts.

Store group membership and observed toll events separately. An ETA can change without rewriting the confirmed checkpoint history.

05 - Data model | Presentation examples | Proposed extension

File: schema/tracking-example.sql

```
-- Simplified proposed tables, not a live schema migration.
CREATE TABLE toll_plazas (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  latitude REAL NOT NULL,
  longitude REAL NOT NULL
);

CREATE TABLE route_segments (
  id TEXT PRIMARY KEY,
  from_plaza TEXT REFERENCES toll_plazas(id),
  to_plaza TEXT REFERENCES toll_plazas(id),
  road_km REAL NOT NULL CHECK (road_km > 0)
);

CREATE TABLE toll_events (
  provider TEXT NOT NULL,
  event_id TEXT NOT NULL,
  trip_id TEXT NOT NULL,
  plaza_id TEXT REFERENCES toll_plazas(id),
  observed_at TEXT NOT NULL,
  received_at TEXT NOT NULL,
  PRIMARY KEY (provider, event_id)
);
```

Explain: the provider and event ID prevent duplicate check-ins. Separate observation and receipt times allow delayed events to be detected. These proposed tracking tables are not connected to a FASTag provider.

SAMPLE OUTPUT - NOT A LIVE SERVER RESPONSE

```
{
  "event_id": "demo-event-003",
  "plaza_id": "plaza-c",
  "observed_at": "2027-01-14T05:55:00Z",
  "received_at": "2027-01-14T05:55:04Z",
  "source": "fictional example"
}
```